

Kazi's AI Agent Playbook

How AI agents plan, build, check and ship any product, with the developer in control

One Claude Code skill that tells an AI coding agent how to work, whether you are starting an empty repo, adding a feature or joining a project halfway through, while keeping you in control of every decision and keeping token use low.

162

checklist items, each with a level, a tag and a runnable check

9

modules: security, QA, AI, legal, UX, performance, ops, SEO, payments

6

hard gates the agent cannot skip without your OK

~2.9k

tokens always loaded (the core); everything else on demand

~5.3k

tokens to start a fresh task (core + one playbook + one module + profile)

#	In this briefing
2	Why it exists: without vs with the guide
3	How the skill works
4	Why it helps you, and how you will notice it
5	How the playbook is organized
6	What happens in every session
7	The two dials: level and flags
8	Hard gates and the task router
9	Phase 0: a new product from nothing
10	Joining an existing project
11	Architecture docs that stay current
12	Feature workflow: idea to ship
13	The checklist modules

#	In this briefing
14	Other stacks: Python, Go, Rails, Laravel
15	How the agent picks skills
16	Which skill for each task
17	Why this way of choosing skills is better
18	Choosing the right model
19	How tokens are saved
20	Making the rules binding
21	Several devs or agents on one repo
22	After launch: monitoring and spend
23	When production breaks
24	Who does what
25 · 26	What to expect · Quickstart and honest limits

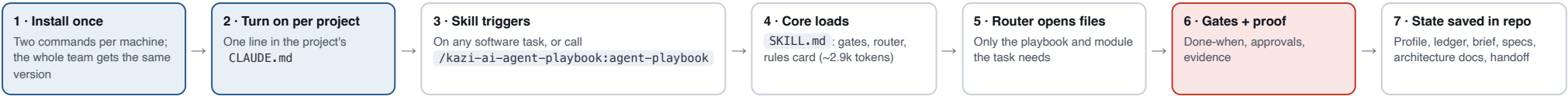
The problems it removes

AI agents are fast but undisciplined by default: they start coding before the problem is clear, re-explore the repo every session, skip checks, claim success without proof and grow long, expensive conversations. The playbook turns those habits into explicit rules.

Area	Without the guide	With the guide
Starting work	Agent writes code from a one-line prompt and guesses the rest.	A "how we'll know it's done" check for every task; an approved brief before a new product and an approved plan before anything users will notice.
Understanding the repo	Re-reads folders and files every session (10-20 exploratory calls).	A 2-second recon script plus a saved project profile. Re-run only when dependencies or config change.
Joining a project mid-way	Agent either treats it like a new product or silently guesses the architecture.	Missing product and architecture docs are drafted from the code, marked for you to correct, and kept current with every change.
Quality	Security, legal, accessibility and ops get remembered only when someone asks.	162 requirements filtered by level and project type; each has a command that proves it.
Honesty of results	"Done" and "looks good" without evidence.	"Fine" needs <code>file:line</code> evidence or command output. Anything not checked is reported as "couldn't check".
Token cost	Huge instructions loaded every time; sessions grow until they drift.	~2.9k-token core; playbooks and modules load only when needed; one task per fresh session with a file handoff.
Skills, tools and models	Agent uses whatever is installed, or nothing; runs every task on whatever model happens to be on.	Proposes a skill set for the product type, falls back when one is missing, asks once before installing; says in one line when a task deserves a different model.
Your control	Agent may push, deploy, add dependencies or install tools on its own.	Installs, deploys, pushes, spending, deleting data, production actions and live legal text always need your approval.
After launch	Problems are found by users; nobody tracks API spend; incidents are handled ad hoc.	A daily first-week review, a weekly spend review against your budget, and one incident flow: contain first, fix with a test, short postmortem.
Teams	Two agents edit the same files and overwrite each other's records.	Tasks are claimed with an owner, each agent works on its own branch or worktree, and shared files merge by rule.
How it talks to you	Jargon and internal labels in every message.	Plain words ("fine / needs fixing / couldn't check", "ready for a free beta"); IDs stay in the files.

How the skill works

The playbook ships as a Claude Code plugin with one skill, `agent-playbook`. You install it once; from then on Claude loads the right rules for each task by itself, in small pieces, so it stays cheap.



Part of the skill	What it is	When it is used
<code>SKILL.md</code> (core)	6 hard gates, level and flags, task router, one-screen rules card, plain-language rules	Every time the skill triggers (about 2.9k tokens)
<code>references/B1...B18</code>	18 playbooks: quickstart, Phase 0, checks, tokens, asking, ledger, sessions and teams, coding, safety, workflow, launch, enforcement, versioning, improving, architecture docs, joining a project, model fit, incidents	Only the one the router names for the task
<code>references/modules/</code>	9 checklists, 162 items, each with a level, a tag and a check command	Only the modules routed by the changed files
<code>references/skill-registry.md</code>	Product type → skills, capability → provider → fallback, skill budget, overlap check	At the start of a session, and for "what should I install?"
<code>references/stack-translations.md</code>	Check equivalents for Python, Go, Rails and Laravel	Only in non-JS/TS projects, once, then saved in the profile
<code>scripts/recon.sh</code>	2-second project scan: stack, flags, routes, tests, missing files	First session in a project, or when config changes. Works with or without git
<code>scripts/gg.sh</code>	Search helper every check runs through	During checks. Uses git when present, plain grep otherwise
<code>assets/profile-template.md</code>	Template for <code>docs/agent-profile.md</code> in your project	Filled once per project, then read each session
Files in your repo	<code>docs/agent-profile.md</code> , <code>docs/readiness-ledger.md</code> , product brief, specs, <code>ARCHITECTURE.md</code> , <code>ARCHITECTURE-ESSENTIALS.md</code> , glossary, plans, incident notes	So any new session continues where the last one stopped

Updates are central: when the playbook improves, the plugin repo gets a new version and everyone runs `claude plugin marketplace update kazi-playbook` then `claude plugin update kazi-ai-agent-playbook@kazi-playbook`, and restarts Claude Code. Nobody copies files between projects.

Why it helps you, and how you will notice it

The skill does not make Claude smarter; it makes Claude disciplined and predictable, which is where most time and tokens are lost. Each benefit below shows up in a specific, visible way.

Why it helps	How you will see it day to day	Evidence from testing
It builds the right thing	New products start with a one-page brief and a stack decision; anything users will notice starts with a short plan. You approve before any code is written.	Feature test: without the skill the agent's first action was writing code; with it, a Done-when and a spec came first. A request the brief listed as out of scope was stopped with one question.
Claims come with proof	"Done" always comes with command output, screenshots or <code>file:line</code> . Anything unchecked is reported as "couldn't check".	Live test: it ran the app's tests and a real browser before reporting; the result was reproduced independently.
You stay in control	Installs, pushes, deploys, production actions, spending and live legal text always wait for your OK, asked once in one batch.	Every run stopped at the approval points; nothing was installed or pushed on its own.
It speaks your language	"I checked 24 things for a free beta: 18 are fine, 4 need fixing", not "PASS 18 / FAIL 4 at L2".	Added after a live test showed gate and item labels leaking into chat (v4.0.5).
Less waste	The core is small; only the needed files open; the project is scanned once and remembered; checks are single greps.	v4.0 measurement: same token cost as without the skill (+2%) for a much higher pass rate. Re-measure on v4.3: the core has grown since.
Quality sized to the stakes	Security, testing, accessibility, legal, ops and SEO checks apply automatically, but only at your project's level.	Readiness test: clear totals and one next action, plus a server-side SSRF risk the baseline missed.
Same result for every teammate	The same task gets the same gates, files and skills whoever runs it; missing skills fall back instead of blocking.	Skill selection resolved against a real session; every skill that was not installed had a stated fallback.
Work survives between sessions	Short sessions with a 10-line handoff file; the next session starts from the profile, plan and architecture docs, not from scratch.	The build step worked from the saved brief and stack decision, and recorded its one change of plan in the decision file.

16 / 16 vs 8 / 16

Checks passed with vs without the skill across 3 realistic tasks (new product, feature, readiness check). One run each on v4.0, so treat it as directional.

Triggers by itself

From a plain "Start Phase 0" prompt, the first thing Claude did was load the skill; no mention of it was needed.

Fixed from real use

v4.0.1 to v4.0.6 came from live tests; v4.1 to v4.3 from a gap review against the full product lifecycle.

How the playbook is organized

Six parts. The agent always reads Part A and loads only the slices Part A routes it to. People can read any part directly.

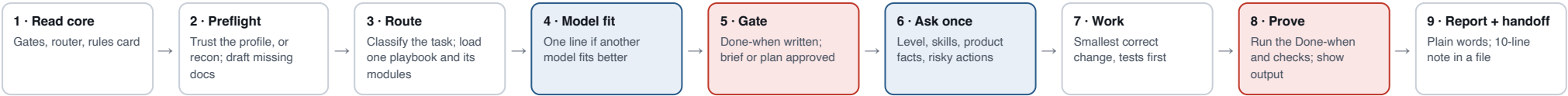
Part	What is inside	Who uses it	When it is loaded	≈ tokens	Changes per project?
A · Core	6 hard gates, level and flags, task router, one-screen rules card, plain-language rules	Agent, every session	Always	2,950	No
B · Playbooks	18 step-by-step procedures (B1...B18) plus the stack-translation tables for non-JS projects	Agent for the current task; you for B1 and B12	One section at a time, when routed	~800 each 15,500 total	No
C · Skill registry	Product type → skills table, capability → provider → fallback, skill budget, how missing skills are requested	Agent at preflight; you when deciding installs	At preflight	2,800	No
D · Modules	9 checklists (162 items): ID, level, tag, requirement, check command	Agent during checks and reviews	Only the routed module	~900 each 8,300 total	No
E · Profile	Level, flags, stack, source dirs, path → module routing, docs inventory, check translations, accepted exceptions, your decisions	Agent fills it once; you confirm	Every session (small)	650	Yes, filled once
F · recon.sh	Read-only script: detects stack, flags, routes, missing files (CSP, privacy, health route, tests, CI)	Agent at first session or when config changes	Run, not read	—	No

Reading in slices is part of the protocol: the router names one file per task, and checks search with one `gg` call, so the agent never pays for the whole ~30k-token skill.

Your repo holds the state: `docs/readiness-ledger.md` (the only place check results are stored, one line per item, a header per run), the filled profile, the product brief, specs with a fixed `**Status:**` line, the architecture docs, plans and the glossary. Any new session can resume from them.

What happens when you give the agent a task

The same loop runs for every task. Red steps are gates; blue steps are where you are involved.



Step	What the agent does	What you see
Preflight	Reads the saved profile; re-runs recon only if manifests, lockfiles, config or CI changed. In an existing repo with no product or architecture docs, drafts them from the code.	Nothing on most sessions. On the first one: a three-line intro, the proposed level and one batched question; a one-line note if docs are being drafted.
Route	Matches the task to the router (new product, feature, bug, UI, review, release, incident...) and the changed paths to modules.	Nothing: routing is silent. You see the work, not the file names.
Model fit	Weights the task's stakes, size, level and open questions against the model you are on.	At most one line, e.g. "This touches payments; I'd switch to a stronger model first." You decide.
Filter	Keeps only items at or below the project level whose tag matches a set flag.	Small diffs get a handful of checks, not 162.
Verify	Runs each item's check with the <code>gg</code> helper; stops at first decisive evidence; subtracts accepted exceptions.	Each result with <code>file:line</code> , recorded in the ledger.
Fix	Fixes failures inside the task's scope and level only; removes code the change left unused; everything else goes to the ledger.	Focused diffs, no drive-by refactors, no orphans.
Prove	Re-runs the failing check plus tests, lint and typecheck; quotes the output.	Evidence, never "should work".
Report	Totals in words, each problem with where and the fix, what it couldn't check, the one next step, decisions waiting on you.	A short report you can act on in a minute.

The two dials: level and flags

Two settings decide which of the 162 items apply. You set the level; the agent proposes the flags from recon and you confirm them. In chat the agent describes the level in words ("a free beta with real users"), not as L1-L4.

Level: how much is at stake

Level	Meaning	Items starting here	Total in force
L1	Prototype or demo, no real user data	11	11
L2	MVP or beta: real users, free	58	69
L3	Public launch or paid product	77	146
L4	Scale or regulated: money at volume, health, minors, enterprise	16	162

"Total in force" is the ceiling; flags remove most of them for any given project.

Flags: what the project contains

Flag	Means	Detected by recon?
auth db api	Sign-in, a database, server routes	yes
ai pay email	LLM calls, payments, sending email	yes
upload render	File uploads, headless browser / PDF rendering	yes
ui public	A user interface, publicly indexed pages	yes
admin	Admin routes or roles	hint
minors lib	Children may sign up; the project is a package	ask you

Example: a free MVP with sign-in and a database (free beta; ui auth db api) never sees payment, email, minors or launch-and-above items. A static landing page (ui public) never sees database or auth items. If the brief and the profile disagree, the profile wins for level and decisions, the brief for product scope.

Hard gates and the task router

Six gates (only you can waive one)

Gate	Rule	What you notice
G1 Done-when	No code edit before a command or observable check is written down.	Each task starts with how you'll both know it works.
G2 New product	No feature code until the brief, stack decision and skeleton plan are approved.	You approve a one-page brief first.
G3 Agree before anything users notice	Decided by visible effect, not diff size: any new button, option, field, page or flow, or a change to data model, auth, money or a public API, needs an approved plan. Small features: 3-6 lines in chat.	A short plan to approve; invisible refactors just proceed.
G4 Ask first	Installs, push, deploy, send, spend, delete data, production actions, live legal text.	One batched question, never repeated.
G5 Evidence	Done means output was seen; unchecked is "couldn't check". A plan is "approved" only if its Status line or you say so.	Command output in every claim.
G6 Precedence	You > project instruction file > this guide > skill defaults. Security floor never traded.	Your repo rules always win.

Task router: you say → the agent loads

You ask for	Agent loads	First thing you get
"Run the preflight"	Check protocol, skill registry	Profile filled, one question
Existing repo, no docs	Joining-a-project + architecture docs	Note on what's being drafted
"Start Phase 0"	New-product playbook + product → skills table	Product brief draft
A new feature	Workflow steps 1-4 + routed modules	Plan or spec for approval
A small visible change	Core only; brief's out-of-scope list first	Out-of-scope question, or a 3-6 line plan
An invisible change	Rules card only	How you'll know it's done
Fix a bug	Diagnose loop	A failing reproduction
UI / design work	UX module and its design-skill rows	Done-when with screenshots
Review this PR	Review + PR steps, routed modules	Findings / PR text
"Run the release check"	Check protocol + all modules at the level	Ledger + plain report
"How did the last check go?"	Ledger format	Latest results, no re-checking
Launch / first week	Launch playbook	Tickets by severity
Something broke in production	Incident response	Severity + containment step to approve
"What should I install?"	Product → skills table + skill budget	What to install and what not to
Long task, context filling	Sessions playbook	Handoff note, fresh session
Hooks or CI	Enforcement playbook	Config for approval

Phase 0: a new product, step by step

For an empty repo or "build me X". Nothing is built until you approve steps 1 and 2; both are drafted together and approved in one round trip. Every step leaves a file, so work can pause and resume in a fresh session.

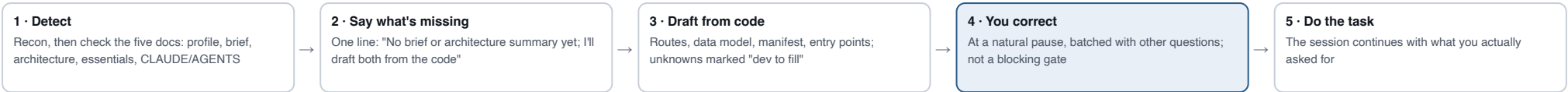
#	Step	Output	Done when	Your role
1	Product brief	<code>docs/PRODUCT-BRIEF.md</code> : one persona, problem and today's workaround, core value loop, MVP must-haves and won'ts, success metric with a target, constraints, monetization, proposed level and flags	You approve it	approve answer the interview
2	Stack decision	<code>docs/decisions/0001-stack.md</code> : 2-3 options, decision, consequences, exit path	You approve it	approve
3	Repo setup	Official starter, lockfile, <code>.env.example</code> , <code>CLAUDE.md</code> pointer, <code>AGENTS.md</code> if the tooling expects one, profile filled, glossary seeded, enforcement baseline	Typecheck, lint, test run green	Review the setup
4	Walking skeleton	Thinnest live slice: sign-in → one real core action → saved → visible, plus CI	Preview URL works, CI green	OK production deploy
4b	Architecture docs	<code>docs/ARCHITECTURE.md</code> and <code>docs/ARCHITECTURE-ESSENTIALS.md</code> , drafted from the stack decision and the now-proven skeleton	You skim and correct	Correct the draft
5	MVP plan	<code>docs/plans/mvp.md</code> : vertical slices, riskiest first (payments, AI quality, data model, third-party limits)	You agree on the order	agree order
6	Build loop	One slice per session through implement → review → PR	Each slice's Done-when seen	Review PRs
7	Launch	Release check at the brief's level, then the launch playbook	Clean, or exceptions you accepted	go / no-go

How the stack is chosen

Criteria, in order	Defaults the agent proposes
1 Team familiarity → 2 fit for the flags → 3 hosting limits and cost → 4 lock-in and exit path → 5 maturity and docs	Boring, well-documented tools · one language end to end · managed services over self-hosting up to a public launch · payment and AI providers confirmed for your country and regions · no microservices, queues or caches before a measured need.

Joining an existing project

Most work happens in repos that already exist. Phase 0 doesn't fit (the product is already running), and guessing the architecture is how agents break things. So on the first session in a repo with no profile or missing docs, the agent builds the same shared picture Phase 0 would, but from the code instead of an interview. It runs quietly inside the preflight, not as a separate ceremony.



Document	How it is drafted	Rule
docs/agent-profile.md	From recon: stack, flags, source dirs, routing; level asked once	Same as any first session
docs/PRODUCT-BRIEF.md	Persona, problem and value loop inferred from UI copy, routes and data model. Success metric, monetization and constraints are marked "unknown, dev to fill", never guessed	Evidence, not invention
docs/ARCHITECTURE.md and ARCHITECTURE-ESSENTIALS.md	From the actual source tree: entry points, data layer, the boundaries that exist today, integrations from the manifest and env. Documents what the code does, not what it should do	A messy boundary is documented as it is, with a note, not tidied up on paper
CLAUDE.md / AGENTS.md	Minimal version only: verified commands, stack, pointer to the essentials doc	ask first These are yours; an existing file only gets the one pointer line

Every drafted doc starts with `<!-- Backfilled from code on <date>; dev: please correct -->`, removed once you confirm it. The profile's `docs:` line records what exists, so the check is not repeated every session. For a big feature or rewrite on top of an unconfirmed draft, the agent asks you to confirm the draft first.

Architecture docs that stay current

Two files with two audiences. A fresh session, human or agent, reads the short one first; the long one is the reference. A stale architecture doc is worse than none, because it misleads the next session, so keeping them current is part of "done".

File	For	Size	Changes when
docs/ARCHITECTURE.md	Deep reference: system overview, components, the main data flows, storage, integrations and their failure modes, non-obvious decisions (linking the decision records), known limits	As long as needed	An architectural decision changes: a new subsystem, a storage change, a new auth model. Not every commit
docs/ARCHITECTURE-ESSENTIALS.md	What anyone must know before touching code: the invariants that are easy to break and expensive to unwind, commands (or a pointer to CLAUDE.md), gotchas already hit once, pointers for more	Under ~200 lines	Every task that touches architecture , in the same commit as the change

The drift rule

Moment	What the agent does
A task changed architecture	Updates ARCHITECTURE-ESSENTIALS.md in the same commit. An architecture change without the doc update is not done.
Drift noticed	The doc says X, the code does Y: fix the doc now, or tell you. Never leave it for later; a later session can't know the doc was already known to be wrong.
End of a long session	The handoff note says whether the essentials doc still matches what was built; if not, fixing it is the first line of the next task.
Your project has its own convention	Your names, format and rules win (for example a CLAUDE.md that says which docs to keep current). The playbook only sets the floor for projects that have none.

Workflow: idea to ship

Small, deliberate steps; each one is scaled to the size of the task and leaves a file. Skill names are examples; each step works without a skill through its fallback.

#	Step	What happens	Output	Skill examples · fallback
1	Align	Interview until every design branch is resolved; vague terms challenged. Skipped for tiny, well-specified changes.	Decisions in the spec	grill-me, brainstorming · question list
2	Shared language	Domain terms defined once and reused in code, UI, tests and tickets.	docs/GLOSSARY.md	domain-modeling · by hand
3	Spec	Small feature: 3-6 lines in chat first. Otherwise problem, users, acceptance criteria, non-goals, checks in scope, risks (blocking ones fixed first), and a <code>**Status:**</code> line: Draft → Approved <date> → Built <commit>.	docs/specs/<feature>.md	to-spec, writing-plans
4	Slice	End-to-end, demoable vertical slices with explicit dependencies; each slice has an owner line.	Plan file / issue tracker	to-tickets · task template
5	Prototype	Optional throwaway to answer one design question, then deleted.	Decision in the spec	prototype
6	Implement	Red → green → refactor for logic; UI verified in a real browser; nothing left unused behind the change.	Code + tests	tdd, implement
7	Diagnose	Reproduce → minimize → hypothesize → instrument → fix → keep the test.	Regression test	systematic-debugging
8	Review	A fresh agent reviews on two axes: standards (checklist, project rules) and spec (did what was agreed, nothing more).	Findings list	code-review, security-review
9	PR	Summary, evidence (commands + output, screenshots), checks touched (re-checked if their files changed), spec set to Built, merge-risk call.	PR description	pr · built-in template
10	Design upkeep	Periodic architecture survey; prefer deep modules with small interfaces; dead-code sweep.	Ranked refactor tickets	improve-codebase-architecture, ponytail-audit
11	Retro	What wasted time or tokens → one concrete change to rules, profile or skills.	1-3 committed lines	retro · by hand

The checklist modules

Every item has an ID, a minimum level, an applicability tag, the requirement, and a check: usually a single grep or command, sometimes a short read. Results land in the ledger with the file paths they depend on, so unchanged items are not re-checked.

Module	Items	Covers	Example requirement → check
sec	34	Secrets, auth, object-level authorization, row-level security, redirects, CSRF, input validation, XSS sinks, rate limits, bot protection, uploads, SSRF, CORS, logs, security headers, supply chain, pentesting at the top level	Every non-public handler authenticates first → list handlers that lack the auth call
qa	16	Run/test commands, unit tests for core logic, API tests (401/400/403/2xx), DB tests, mocking, CI gate, E2E critical path, visual regression, migrations	CI required on the default branch → workflow file + branch protection
ai	16	Keys server-side, prompt-injection delimiting, output caps, per-user budgets, spend rolled up against a monthly budget, structured output, retries, data minimization, evals, disclosure	Every model call has an output cap → grep for the max-tokens parameter
legal	15	Privacy policy and terms, data minimization, account deletion/export, consent only when needed, subprocessors, honest claims, licenses, placeholder content, pricing transparency	No fake testimonials or invented stats → grep marketing copy for claims
ux	25	Design tokens, reuse, loading/empty/error states, responsive, keyboard, forms, contrast, accessible names, motion rules, Figma-to-code rules	Animate only transform and opacity → grep transitions on layout properties
perf	15	Bundle size, indexes, N+1 queries, pagination, pooled connections, timeouts, debouncing, code splitting, images and fonts, Core Web Vitals, caching only after measurement	LCP < 2.5s, INP < 200ms, CLS < 0.1 on top pages → Lighthouse
ops	15	README, rollback, backups and restore drills, free-tier limits, health endpoint, error tracking, logs, uptime, migrations, cost guards, runbook (linked to the incident flow)	Backups exist when the tier has no point-in-time recovery → ops doc / workflow
seo	15	noindex on app routes, titles and descriptions, sitemap/robots, canonical, headings, Open Graph, honest JSON-LD, real 404s, favicons, answer-engine structure	Custom 404 returns HTTP 404 → curl -I a missing URL
pay	11	Hosted checkout, webhook signature on raw body, idempotency, server-side entitlements, sandbox end-to-end, dunning, tax, commercial-use hosting	Webhook replays are no-ops → unique event-id constraint

Known intentional matches go into the profile's accepted exceptions, so they are not re-triaged each session. Checks written for JavaScript/TypeScript have worked equivalents for other stacks (next page).

Other stacks: Python, Go, Rails, Laravel

Most checks are written as JavaScript/TypeScript searches. For other stacks the agent uses ready-made equivalents in `references/stack-translations.md` instead of inventing ~100 translations on the spot, and copies each one into the profile the first time it is used.

	Python	Go	Ruby (Rails)	PHP (Laravel)
Test	<code>pytest</code>	<code>go test ./...</code>	<code>bin/rails test / rspec</code>	<code>php artisan test</code>
Typecheck / lint	<code>mypy / pyright · ruff</code>	<code>go vet · staticcheck</code>	<code>rubocop</code>	<code>phpstan</code>
Dependency audit	<code>pip-audit</code>	<code>govulncheck</code>	<code>bundle-audit</code>	<code>composer audit</code>
Dead code	<code>vulture</code>	<code>staticcheck (U1000)</code>	<code>debride</code>	<code>phpstan</code>

Security and test checks with worked equivalents

Check	What the translation looks for (examples)
Secrets never reach the browser	Templates printing <code>SECRET</code> / <code>_KEY</code> / <code>TOKEN</code> values; the JS check on any separate frontend
Identity is verified	Unverified JWT decoding: <code>verify_signature: False</code> , <code>ParseUnverified</code> , <code>JWT.decode(t, nil, false)</code>
Every handler authenticates	<code>login_required</code> / <code>Depends(get_user)</code> , router auth groups, <code>before_action :authenticate</code> , <code>auth</code> middleware
Queries filter by owner	<code>.objects.get(...)</code> without <code>user=</code> , <code>Model.find(params...)</code> instead of <code>current_user.things.find</code> , <code>unscoped findOrFail</code>
Input validated, no string-built SQL	Serializers / <code>pydantic</code> / <code>permit</code> / <code>FormRequest</code> ; f-strings, <code>Sprintf</code> , <code>#{}</code> or <code>\$var</code> inside SQL calls
No unsafe HTML, no PII in logs, services mocked, real error pages	<code> safe</code> , <code>template.HTML</code> , <code>html_safe</code> , <code>{!!}</code> ; logger calls with token/password/email; <code>@patch</code> , <code>httptest</code> , <code>webmock</code> , <code>Http::fake</code> ; 404/500 templates

Calibrate before trusting "fine". The first time a translation runs, the agent points it at one file that should match. A pattern that silently matches nothing would report a problem as fine, the worst result a check can give. All 14 security patterns were tested against sample files; none has yet run on a real Python, Go, Rails or Laravel project. Other stacks (Java, .NET, Rust, Elixir...) are translated the same way and saved in the profile.

How the agent picks skills

The agent proposes skills for your product type, checks what is actually installed, uses a built-in fallback when something is missing, and asks you once. It never installs anything itself.

Product type	Flags · level	Modules loaded	Skills it proposes (capabilities)
Any new product	from the brief	per the row below	One workflow pack (align, spec, slice, TDD, debug, review) · code review
Landing / marketing	ui public · L2-L3	ux, seo, perf, legal(01,09), sec(50)	Design-quality skill · one style preset · animation audit only if motion matters · browser tool
Frontend app	ui api · L2	ux, qa, perf, sec(30-39)	Design-quality skill · browser tool · TDD
MVP SaaS (free)	ui auth db api · L2	sec, qa, ux, ops(01-03,05), legal(01-03)	Workflow pack · security review · browser tool · docs lookup
Paid SaaS	+ pay email · L3	all	+ payment-provider docs · security review on every money PR
AI-powered app	+ ai · L2-L3	+ ai	AI API/SDK skill · golden-set evals · spend review
API / backend	api db auth · L2-L3	sec, qa, perf, ops	TDD · security review · code-graph tool for large codebases
Internal tool	auth db ui · L2	sec, qa, ux	Workflow pack (no style presets)
Library / package	lib · L2-L3	qa(01-03,07), sec(51,52), ops(01,02)	TDD · code review · docs-writing skill

Skill budget (installed skills cost tokens in every session)

Rule	Why
One workflow pack	Packs overlap on planning, TDD, debugging and review.
At most one style preset	Presets conflict; stacking gives muddled design.
At most one animation audit	Enough to catch motion problems.
Task-only skills removed after	Prototype, library picker, brand, slides.
Your tokens and components win	No style skill overrides the design system.

What a missing-skill question looks like

Animation audit: improve-animations not installed.
Unlocks: audits every animation for purpose, timing, reduced motion.
Without it I will: apply the motion rules and audit steps by hand.
To install (you run it): /plugin install <plugin>@<marketplace>
Choose: [I'll install it, wait] [Use fallback] [Skip this area]

Asked once per session, at most four per prompt; no answer means "use fallback"; the answer is saved in the profile so it is never asked again.

Which skill the agent picks for each task

One skill per capability: the first one actually installed in the session. If none is, the built-in fallback keeps work going. Names are examples from common packs; equivalent skills count.

Task	Skill the agent picks	If it is not installed	Why this skill
New product or feature	grill-me / brainstorming → to-spec → to-tickets	Question rounds, then the brief, spec and task templates	Every open question answered before code; small vertical slices.
Domain terms	domain-modeling	Hand-written GLOSSARY.md	Same words in code, UI and tickets.
Design question	prototype	One throwaway page or route, deleted after	Answers the question before real code depends on it.
Writing logic	tdd / test-driven-development	Failing test → fix → run tests, by hand	Proof it works, and a test that catches regressions.
Writing lean code	ponytail	The "smallest correct change" ladder by hand	Reuse, standard library and platform features before new code.
Fixing a bug	systematic-debugging	Reproduce → minimize → hypothesize → fix loop	Root cause, not symptom; stops blind retry loops.
Build or polish UI	impeccable (design quality)	UX module rules + existing components	Layout, states, accessibility and polish in one pass.
Landing-page look	ONE style preset (taste / minimalist / premium)	Existing design tokens	A distinctive look without conflicting styles.
Redesign or animation	redesign-existing-projects · one animation audit	Small diffs with screenshots · motion rules by hand	Better look without breaking behavior; purposeful motion.
Charts / dashboards	dataviz	Chart library defaults + UX rules	Readable, accessible, consistent charts.
Check UI in a browser	Playwright (browser tool)	Script, else UI items "couldn't check"	Real screenshots, keyboard and mobile checks.
Debug a live page	Chrome DevTools MCP	Playwright console/network tools; or you paste output	Console errors, network waterfall, performance traces.
AI / LLM feature	claude-api (or the provider's SDK skill)	Provider's current docs; flagged "unverified"	Correct model IDs and request rules.
Library API question	context7 (docs lookup)	The installed package's own docs	Version-accurate answers, not memory.
"Who calls this?"	codebase-memory (code graph)	gg + short reads	Structural answers without reading whole files.
Code review	code-review, by a fresh agent	Diff checked against the QA module	Fresh eyes on standards and the spec.
Security review	security-review	Security module checks via grep	Auth, data-leak and injection issues before merge.
Try to exploit it (highest level)	An agentic pentest tool, e.g. Strix-class	Static checks only, and the report says so	Real attack attempts, on a non-production copy, with your OK.
Simplify / remove bloat	simplify / ponytail-review / ponytail-audit	"Can this be deleted?" pass on the diff	Less code to maintain.
Dead code	knip / ts-prune / vulture, code graph	Grep the removed name across the source	A removal or rename never leaves orphans.
Before saying "done"	verification-before-completion	Run the Done-when; quote the output	No success claims without evidence.
Big plan, many tasks	writing-plans → subagent-driven-development	Plan file + one fresh session per task	Small focused contexts; cheaper, less drift.
Handoff · retro · hooks	handoff · retro · update-config	10-line note · lessons by hand · exact settings for you	Clean restarts, rules that improve, rules made automatic.

Why this way of choosing skills is better

Without rules, an agent uses whatever skill it notices first, several at once, or none. The guide makes the choice predictable, cheap and always under your control.

Rule in the guide	What usually happens without it	Why the guide's way is better
Pick from the task, not from habit	The agent reaches for a familiar skill even when the task needs a different one.	The router and the table on the previous page give the same answer every time, across sessions and teammates.
Only skills actually installed	The agent "calls" a skill that does not exist in this setup, or describes one from memory.	No invented skills; the session's own list is the only source of truth.
One skill per capability	Two overlapping skills give conflicting advice (two style presets, two workflow packs).	One clear direction; no muddled design or duplicated steps.
Capabilities, not brand names	A gap stays open because the one named tool isn't installed.	Any installed tool with the capability counts, so new tools slot in without changing the playbook.
Built-in fallback for every row	Work stops, or quality drops, when a skill is missing.	Work never blocks; the fallback applies the same rules by hand.
Ask once, batched	The agent interrupts you repeatedly about tools.	One question per session, max four items; the answer is saved and never asked again.
Never install on its own	Tools get added silently; nobody knows what is running.	You decide what is installed; the setup stays reviewable.
Skill budget	Dozens of skills installed; their descriptions load into every session.	Fewer, non-overlapping skills: fewer tokens per session and fewer wrong picks.
Product type decides the set	A landing page gets backend tooling; an API gets design presets.	Each product type gets the skills that matter for it.
Your rules beat skill defaults	A skill's advice overrides the project's design system or conventions.	Project instruction file and design tokens always win.
Net effect for the team: the same task gets the same tools no matter who runs it. Missing tools never block work. Your skill list stays small and deliberate, and every addition is your decision.		

Choosing the right model

A hard task on a weak model gets redone; a typo fix on the strongest model wastes budget. Once per fresh session, before the first edit, the agent compares the task with the model you are running. It cannot switch models itself (only you can, with `/model``), so this is one line of advice, never a block.

What decides the recommendation

Signal	Points to a stronger model when...
What the task touches	A new product, or anything users will notice that changes the data model, auth, money or a public API
Size	More than ~5 files or more than one area of the code
How strict the project is	A public or paid launch and above: never a budget model, however small the diff
Open questions	More than ~2 unresolved design choices: ambiguity costs judgment, not size

Signals combine by the worst case: any one of them calling for the stronger model wins. Trivial invisible changes (a typo, a dependency bump) skip the check.

What you hear

"This touches auth and the data model, so I'd recommend switching to a stronger model before I start; mistakes here are expensive to redo. If you want to continue on the current model, say so and I will."

A cheaper-model suggestion is only offered when you ask about cost. If you don't answer, the agent carries on with the current model. Your choice is saved for the session and checked again in the next fresh session.

Separate rule for helpers

Sub-agents the playbook starts for searching and sweeping run on the cheapest model that can grep. Security fixes, architecture decisions and ambiguous failures always go to the strong model.

How tokens are saved

Every message resends the whole conversation, so the biggest savings come from loading less, exploring less and keeping sessions short.

Technique	How it works	What it avoids
Small always-loaded core	Only the core (~2.9k tokens) is read every session; playbooks and modules load on demand.	Paying for the whole ~30k-token skill on every task.
Task router	Each task type maps to exactly the sections it needs.	Loading security and payment rules for a CSS tweak.
Level + flag + path filters	Only items at or below the level, matching set flags and changed paths.	Running 162 checks on a small diff.
Recon once, profile reused	A 2-second script; the profile is trusted until config or dependencies change. Missing docs drafted once, not re-explored.	10-20 exploratory calls per session.
Grep before read	Checks are single greps over source files; reads limited to ±15 lines.	Reading whole files to "understand" them.
Ledger with paths	A result is reused until a file it depends on changes; "how did the last check go?" reads the ledger.	Re-verifying unchanged items.
Accepted exceptions	Known intentional matches are recorded once.	Re-triaging the same false hits every session.
Cheap sub-agents for sweeps	Cross-file sweeps run on a small model and return a 15-line table.	File dumps in the main conversation.
Right model per task	A one-line model recommendation before work starts.	Top-model prices for small work, or redoing hard work done on a weak model.
One task per fresh session	Fresh session when the task changes or context is about half full; a 10-line handoff file carries state.	Long sessions that grow expensive and drift.
Gates before code	Done-when and an approved brief or plan first.	Rework, the largest hidden token cost.
Skill budget · short reports	Few, non-overlapping skills installed; reports give totals, problems and one next step.	Skill descriptions loaded for nothing; long summaries nobody needs.

Starting a fresh task costs about 5.3k tokens of guide: core (~2.9k) + one playbook section (~0.8k) + one module (~0.9k) + profile (~0.65k). Optional: record tokens: in each handoff note to see where cost leaks.

Enforcement by level

A guide is advice; agents can still skip a rule. Hooks and CI make the important rules mechanical. Turn on your level's row and every row above it.

Level	Enforce with	What it catches
L1	Formatter and linter in the editor; <code>typecheck</code> , <code>lint</code> , <code>test</code> scripts exist	Style drift, obvious type errors
L2	Pre-commit hook on staged files: format, lint, typecheck, secret scan · CI on every PR: frozen install → typecheck → lint → test → build, required to merge	Broken builds, leaked secrets, failing tests reaching the main branch
L3	+ dependency audit and Dependabot/Renovate · E2E smoke test on the preview deploy · branch protection with one review · host-side secret and code scanning	Vulnerable dependencies, broken critical path, unreviewed merges
L4	+ SAST/DAST, bundle and performance budgets in CI, signed releases, periodic pentest	Security and performance regressions, supply-chain tampering
Agent hooks	In the agent tool's project settings (e.g. Claude Code <code>.claude/settings.json</code>): after each edit run formatter + typecheck on that file; when a task stops run its narrow tests. Set up with your approval.	What the agent forgets while it works, before it reaches CI

The guide includes a minimal copy-paste CI workflow (GitHub Actions) that you adapt to the project's language and scripts.

Several devs or agents on one repo

The playbook keeps its state in shared files (profile, ledger, specs, architecture docs). Two agents writing them at once would overwrite each other, so working in parallel follows four rules.

Rule	How it works
Claim before starting	Each task block in the plan file has an <code>Owner: line (<dev or agent> · <branch> · since <date>)</code> . Pick only a task whose owner is empty, fill it in and commit that line before writing code. Clear it at handoff. A claim with no commits for 2+ days is stale: ask before taking it over.
One branch per task, one working copy per agent	Two agents on the same machine each get their own git worktree. Never two agents on the same files at once; if two tasks need the same files, finish one first.
Shared files merge by rule	Ledger: one line per item, newest on top, so on a conflict keep both sides' lines and never delete one. Profile and essentials doc: change only your lines, in a commit with the reason; on a conflict keep the newer verified date and re-run recon if unsure. Specs: one file per feature, never shared.
Before merging	Update from the default branch, re-run the narrow tests, and re-check the ledger lines whose files someone else's merged change touched.
Task: Add CSV export to the reports page Owner: Kazi's agent · feat/csv-export · since 2026-10-05 Done when: pnpm test reports passes and the downloaded file opens in Excel with 3 columns Files: app/reports/*, lib/export/* Handoff: docs/plans/reports.md	

After launch: monitoring and spend

Launch is where most guides stop. The playbook keeps going with a short routine, sized to the level like everything else.

When	What happens
Before launch	Release check at the brief's level; every failure fixed or accepted by you · rollback tested once · analytics and consent decided · support contact visible · backups confirmed · paid-API budget recorded and spend visible. Monitoring follows the level: error tracking and uptime alerts are required for a public or paid launch, and recommended for a free beta, where at minimum someone reads the host's error logs daily in week one.
Launch day	Deploy in working hours with someone watching · no unrelated merges · watch errors and the core flow · if the core flow or payments break, the incident flow (next page): contain first, debug second.
First week, daily (10 min)	Top 5 new errors · the core-flow funnel against the brief's success metric · yesterday's paid-API spend against the daily share of the budget · user feedback · every issue becomes a ticket with a severity. Severity 1 (data loss, security exposure, payments broken, core flow broken) is fixed the same day.
End of week one	Metric vs target · retro · brief updated with what was learned · plan re-ordered.
Weekly	Dependency updates and audit · spend review if the product calls paid APIs (AI, email, rendering, storage): total this week, cost per active user, top 5 users by cost, against your monthly budget. A jump of more than 50% week over week, or a projected overrun, becomes a ticket.
Monthly · every few weeks	Backup restore check for a public or paid product · architecture and dead-code survey.
Before each major release	Re-run the release check at the current level.

Why spend gets its own line: for an AI product, model calls are usually the biggest variable cost. Per-call logging and per-user caps were already checks; the playbook now also rolls spend up against a budget you set, and asks you for that budget up front.

When production breaks

One path from "something is wrong" to "it won't happen again". Rule zero: contain first, debug second. The agent never acts on production alone, even for the worst incident; it makes your "yes" fast instead.

#	Step	What the agent does	Your role
1	Assess (≤ 5 min)	Read-only: what users see, since when, which deploy, how many users. Severity: S1 data loss, security exposure, payments or core flow broken · S2 degraded but usable · S3 cosmetic. Looks up the runbook.	—
2	Contain	Proposes the smallest step that stops the harm, in order: turn the feature off (kill switch) → roll back to the last good deploy → disable the broken route. Leaked key: revoke and rotate at the provider first. One message: what's broken, the step, what it undoes, the exact command, ready to run.	approve always
3	Verify	Health endpoint answers, error rate back to normal, core flow works end to end; shows the output.	—
4	Fix forward	On a branch: failing test that reproduces it → fix → keep the test. Normal review and CI; a hotfix skips CI only if you say so.	approve deploy
5	Tell users	If data or security was involved, drafts the notice. Legal notification duties are yours or a lawyer's. Sends nothing itself.	approve
6	Postmortem	≤ 10 lines, blameless, in docs/incidents/<date>--<slug>.md : what users saw, timeline, root cause, what stopped it, what prevents a repeat. Updates the runbook; proposes a new check if one would have caught it.	Review

"Payments are failing for everyone since the 14:02 deploy. I recommend rolling back to the 13:40 deploy now, which undoes today's checkout change. Command: vercel rollback <url>. Shall I run it?"

Severity sets the pace: S1 → steps 1-3 now and the fix the same day · S2 → contain if cheap, otherwise fix next session · S3 → ticket only.

Who does what

The agent does the work and the checking. You keep every decision that is about money, risk, the outside world or the product's direction.

Activity	Agent	You (developer)
Project level	Proposes one from the brief	decide
Flags and profile	Detects with recon, fills the profile	Confirm; add minors , lib
Product brief, stack, plans and specs	Interviews you, drafts the documents	approve
Docs for an existing project	Drafts brief and architecture docs from the code	Correct the draft
CLAUDE.md / AGENTS.md	Drafts only when you agree; adds a pointer line otherwise	Own them
Plan and slice order	Slices work, riskiest first	agree
Code, tests, checks	Writes, runs, proves with output	Review PRs
Model for the session	Recommends in one line	Switch, or carry on
Skills and tools	Proposes, uses fallbacks, asks once	Install or decline
New dependencies	Justifies size, license, maintenance, audit	approve
Push, deploy, send, spend, delete data	Never on its own	approve each time
Production incident	Assesses, proposes containment with the exact command, fixes forward, writes the postmortem	approve each action
Legal text (privacy, terms, refunds)	Drafts and wires it in	approve (ideally with a lawyer)
Outside accounts and budget	Lists the exact steps; tracks spend	Create and connect them; set the budget
Accepted exceptions	Proposes with a reason	approve
Launch go / no-go	Runs the release check and reports	decide
Improving the guide	Suggests changes from retros	Accept, bump the version

What to expect when you start using it

Timeline of the first sessions

When	What happens
Session 1	A three-line intro, recon, proposed level, one batched question (level, missing skills, business facts, API budget). Profile saved.
Existing repo	Missing brief and architecture docs drafted from the code, marked for you to correct; then your task.
New product	Brief interview and draft plus stack decision → your approval. No product code yet.
First build	Repo setup and a walking skeleton on a preview URL with CI green; architecture docs drafted.
Each task after	Model-fit line if needed, Done-when first, small diff, evidence, plain report, handoff note. Fresh session for the next task.
After 3 tasks	Calibration retro: fix noisy checks, record accepted exceptions and translations.
Before launch	Release check at the project level; every failure fixed or accepted by you.
Launch week	Daily 10-minute review of errors, the core flow, spend and feedback; severity-1 issues fixed the same day.

A typical check report, in chat (illustrative)

I checked 26 things for a free beta: 21 are fine, 3 need fixing, 2 I couldn't check.

Needs fixing:

- The export route has no rate limit, so one user can run it in a loop and drive up costs; add the shared limiter (app/api/export/route.ts:1) (SEC-33).
- The API routes have no tests for signed-out, bad input or someone else's data; add table tests (app/api/*) (QA-04).
- There is no health endpoint, so an uptime monitor can't tell the database is down; add a thin route that pings the DB (OPS-05).

Couldn't check: upload size limits (the parser config wasn't readable) and backups (I need to know your database hosting tier).

Next step: add the rate limiter to the export route.

Decision for you: does your database plan include backups? Without them a bad migration can't be undone.

Handoff note between sessions

Changed: app/api/export/route.ts, lib/rateLimit.ts
Checks: SEC-33 fine (rate limit at route.ts:14); tests 48/48
Left: QA-04 route tests
Architecture essentials: still match
Decisions: none open · Gotcha: limiter key must use user id when signed in

Quickstart and honest limits

Set up in about two minutes

#	Do this
1	Install once: <code>claude plugin marketplace add kaziragib18/kazi-ai-agent-playbook</code> then <code>claude plugin install kazi-ai-agent-playbook@kazi-playbook</code> .
2	Add one line to the project's <code>CLAUDE.md</code> : use the <code>agent-playbook</code> skill for every task and ask before installing anything.
3	Say "Run the preflight" (existing repo), or "Start Phase 0" (brand-new product).
4	Answer the one batched question; answers are saved in <code>docs/agent-profile.md</code> .
5	Write tasks with an outcome, how you'll know it's done, and the files in scope.
6	For a release, say "Run the release check" (add "for a public paid launch" to set how strict).
7	Something broke in production? Say so plainly: "payments are failing since the last deploy".
8	For a free beta and above, turn on the pre-commit hook and required CI.
9	Update: <code>claude plugin marketplace update kazi-playbook</code> , then <code>claude plugin update kazi-ai-agent-playbook@kazi-playbook</code> , then restart. Check with <code>claude plugin list</code> .

Treat v4.3 as a tested baseline with new, unpiloted parts. It becomes stable after a pilot on two or three different project types (for example a new MVP, an existing API in another language, and a production incident drill), recorded in the guide's changelog.

What it does not do

Limit	What to do about it
It is advice, not enforcement	Use the <code>CLAUDE.md</code> line plus hooks and CI (page 20).
Grep checks are hints, not proof	They find missing things well; subtle logic bugs still need tests and review.
Newest parts not yet piloted	Joining existing projects, architecture docs, model fit, team rules and incident response (v4.1-4.3) have not run on real projects yet. Report what goes wrong.
Other stacks lightly tested	Translations are tested on sample files only; the first run on a real Python, Go, Rails or Laravel repo calibrates them.
Token result needs re-measuring	The +2% token result on page 4 was measured on v4.0; the core has grown since (~2.7k → ~2.9k tokens). Re-measure with the token-comparison script.
Legal items are drafts	A human, ideally a lawyer, approves anything that goes live.
Skill names vary by pack	The agent uses the skills actually installed and falls back when a name is missing.
Needs a real level from you	A wrong level means too many or too few checks; revisit it at each launch stage.